1. Write a Bash script to check whether a directory exists in Linux.

```
if [ -e /home/data]
 then
        echo "Directory exists"
else
        echo "Directory does not exist"
fi
~
~
```

if checks the specified condition.

-e checks whether the given path exists.

then executes the command if the condition is true.

echo displays the appropriate message.

else executes when the condition is false.

fi ends the conditional statement.

Execution Steps:

Open the file using vi is_directory.bash.

Press Esc, type :wq, and press Enter to save and exit.

Give execution permission using chmod 755 is_directory.bash.

Run the script using ./is_directory.bash.

```
[ec2-user@ip-172-31-43-150 bash_scripts]$ vi is_directory.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ chmod 755 is_directory.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./is_directory.bash
Directory exists
[ec2-user@ip-172-31-43-150 bash_scripts]$
```

2. Create a Bash script to create multiple files in Linux.

```
for i in {1..5}
do
    touch "file$i.txt"
done
~
~
~
```

Explanation of the Code:

for starts a loop to repeat the commands.

i in {1..5} generates numbers from 1 to 5.

do begins the commands inside the loop.

touch creates empty files.

file$i.txt creates files named file1.txt to file5.txt.

done ends the loop.

Execution Steps:

Open the file using vi create_multiple_files.bash.

Enter the script shown in your screenshot.

Press Esc, type :wq, and press Enter to save and exit.

Give execution permission using chmod 755 create_multiple_files.bash.

Run the script using ./create_multiple_files.bash.

Verify the files using ls.

```
[ec2-user@ip-172-31-43-150 bash_scripts]$ vi create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ chmod 755 create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ls
create_multiple_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  is_directory.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$
```

3. Create a bash script to take a backup of a directory.

```
#!/bin/bash

echo "Enter the main directory path:"
read main_directory

echo "Enter the backup directory path:"
read backup_directory

cp -r "$main_directory"/. "$backup_directory"/

echo "Backup completed successfully!"
~
~
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. echo displays messages on the screen prompting the user for input.
3. read captures user input from the keyboard and stores it into variables (main_directory and backup_directory).
4. cp -r stands for copy recursively, which allows copying folders along with all the files and subfolders inside them.
5. "$main_directory"/. targets all the contents inside the source directory (including hidden files).
6. "$backup_directory"/ specifies the destination folder where the files will be backed up.

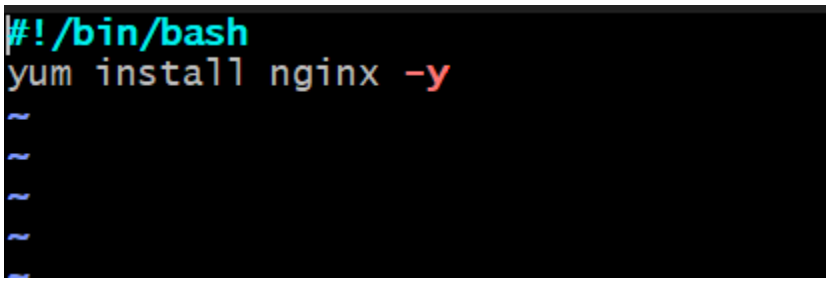
Execution Steps:

1. Open the file using vi backup_dir.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Create a new folder for your backup files using mkdir backup.
5. Give execution permission to the script using chmod 755 backup_dir.bash.
6. Run the script using ./backup_dir.bash.
7. When prompted, enter /home/ec2-user/bash_scripts as the main directory path, and /home/ec2-user/backup as the backup directory path.
8. Verify the files were copied correctly into the new folder using ls backup.

```
[ec2-user@ip-172-31-43-150 bash_scripts]$ vi create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ chmod 755 create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./create_multiple_files.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ls
create_multiple_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  is_directory.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$vi backup_dir.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ mkdir backup
[ec2-user@ip-172-31-43-150 bash_scripts]$ chmod 755 backup_dir.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./backup_dir.bash
Enter the main directory path:
/home/ec2-user/bash_scripts
Enter the backup directory path:
/home/ec2-user/bash_scripts
cp: '/home/ec2-user/bash_scripts/.' and '/home/ec2-user/bash_scripts/.' are the same file
Backup completed successfully!
[ec2-user@ip-172-31-43-150 bash_scripts]$ ls
backup  backup_dir.bash  create_multiple_files.bash  file1.txt  file2.txt  file3.txt  file4.txt  file5.txt  is_directory.bash
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./backup_dir.bash
Enter the main directory path:

Enter the backup directory path:
^C
[ec2-user@ip-172-31-43-150 bash_scripts]$ ./backup_dir.bash
Enter the main directory path:
/home/ec2-user/bash_scripts
Enter the backup directory path:
/home/ec2-user/backup
Backup completed successfully!
[ec2-user@ip-172-31-43-150 bash_scripts]$
```

4. Create a bash script to install nginx.



Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. yum install nginx -y runs the package manager command to install the Nginx web server, where the -y flag automatically answers "yes" to all installation prompts.

Execution Steps:

1. Open the file using vi install_nginx.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 install_nginx.bash.
5. Run the script using ./install_nginx.bash.

```
[root@ip-172-31-43-150 bash_scripts]# vi install_nginx.bash
[root@ip-172-31-43-150 bash_scripts]# chmod 755 install_nginx.bash
[root@ip-172-31-43-150 bash_scripts]# ./install_nginx.bash
Last metadata expiration check: 1 day, 11:43:43 ago on Wed Oct 14 22:45:33 2026.
Package nginx-1:1.30.5-1.amzn2023.0.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-43-150 bash_scripts]#
```

5. Create a bash script to **install Apache Tomcat on an EC2 server**.

```
#!/bin/bash
wget https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.27/bin/apache-tomcat-11.0.27.tar.gz

tar -xzf apache-tomcat-11.0.27.tar.gz

echo "Tomcat downloaded and extracted successfully!"

~
~
~
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. wget apache.org downloads the Apache Tomcat archive file from the official Apache distribution website using the network downloader utility.
3. tar -xzf apache-tomcat-11.0.27.tar.gz extracts the downloaded compressed gzip tarball archive file into the current directory.
4. echo "Tomcat downloaded and extracted successfully!" prints a success confirmation message on the terminal window.

Execution Steps:

1. Open the file using vi install_tomcat.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 install_tomcat.bash.
5. Run the script using ./install_tomcat.bash

```
[root@ip-172-31-43-150 bash_scripts]# vi install_tomcat.bash
[root@ip-172-31-43-150 bash_scripts]# chmod 755 install_tomcat.bash
[root@ip-172-31-43-150 bash_scripts]# ./install_tomcat.bash
--2026-10-16 10:42:18--  https://dlcdn.apache.org/tomcat/tomcat-11/v11.0.27/bin/apache-tomcat-11.0.27.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14545574 (14M) [application/x-gzip]
Saving to: 'apache-tomcat-11.0.27.tar.gz'

apache-tomcat-11.0.27.tar.gz              100%[=============================================================

2026-10-16 10:42:20 (379 MB/s) - 'apache-tomcat-11.0.27.tar.gz' saved [14545574/14545574]

Tomcat downloaded and extracted successfully!
[root@ip-172-31-43-150 bash_scripts]#
```

6. Create a bash script to check if the Nginx service is running, if not running then script should start the service.

```
#!/bin/bash

systemctl is-active --quiet nginx
status=$?

if [ "$status" -eq 0 ]
then
    echo "Nginx is running."
else
    echo "Nginx is not running. Starting Nginx..."

    sudo systemctl start nginx

    systemctl is-active --quiet nginx
    status=$?

    if [ "$status" -eq 0 ]
    then
        echo "Nginx started successfully."
    else
        echo "Failed to start Nginx."
    fi
fi
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. systemctl is-active --quiet nginx checks the active operational status of the Nginx service without printing any output to the screen.
3. status=$? captures the exit code of the previous status check command and saves it into a variable named status.

4. if ["$status" -eq 0] opens a conditional block to evaluate if the exit status code equals 0, which signifies that the service is already running normally.
5. then echo "Nginx is running." specifies the message block to print if the active status condition is met successfully.
6. else echo "Nginx is not running. Starting Nginx..." handles the alternative scenario where the service is inactive, alerting the user that a start process is beginning.
7. sudo systemctl start nginx executes the system command with administrative privileges to start up the Nginx web server service.
8. systemctl is-active --quiet nginx runs a second quiet status check to verify the result of the start command.
9. status=$? saves the exit code of the second service verification attempt into the status variable.
10. if ["$status" -eq 0] runs an internal nested evaluation to see if the service successfully transitioned to an active state.
11. then echo "Nginx started successfully." outputs a positive confirmation text if the service initialized without errors.
12. else echo "Failed to start Nginx." acts as an error reporting backup text if the systemctl start command encountered issues.
13. fi closes the inner service verification conditional statement block.
14. fi closes the main outer service state validation conditional statement block.

Execution Steps:

1. Open the file using vi nginx_status.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 nginx_status.bash.
5. Run the script using ./nginx_status.bash

```
[root@ip-172-31-43-150 bash_scripts]# vi nginx_status.bash
[root@ip-172-31-43-150 bash_scripts]# chmod 755 nginx_status.bash
[root@ip-172-31-43-150 bash_scripts]# ./nginx_status.bash
Nginx is not running. Starting Nginx...
Nginx started successfully.
[root@ip-172-31-43-150 bash_scripts]#
```

7. Create a bash script for a calculator.

```
#!/bin/bash

echo "Enter the first number:"
read num1

echo "Enter the second number:"
read num2

echo "Choose an operation:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

read choice

case $choice in
    1)
        result=$((num1 + num2))
        echo "Result = $result"
        ;;
    2)
        result=$((num1 - num2))
        echo "Result = $result"
        ;;
    3)
        result=$((num1 * num2))
        echo "Result = $result"
        ;;
    4)
        if [ "$num2" -eq 0 ]
        then
            echo "Error: Division by zero is not allowed."
        else
            result=$((num1 / num2))
            echo "Result = $result"
        fi
        ;;
    *)
        echo "Invalid choice!"
        ;;
esac
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. echo "Enter the first number:" displays a message on the screen prompting the user to type in their initial numeric value.

3. read num1 captures the user input from the keyboard and stores it into a variable named num1.
4. echo "Enter the second number:" displays a message prompting the user for their secondary numeric value.
5. read num2 captures the second input and stores it into a variable named num2.
6. echo commands display a numbered menu list of mathematical operations including Addition, Subtraction, Multiplication, and Division.
7. read choice captures the user's menu selection number and stores it in a variable named choice.
8. case $choice in starts a conditional control block that matches the user's choice against the defined menu numbers.
    4. evaluates the addition operation using the arithmetic expansion syntax $((num1 + num2)) and prints the result.
    5. evaluates the subtraction operation using the arithmetic expansion syntax $((num1 - num2)) and prints the result.
    6. evaluates the multiplication operation using the arithmetic expansion syntax $((num1 * num2)) and prints the result.
    7. handles the division operation and opens an internal if condition to verify if the second number equals 0.
9. if ["$num2" -eq 0] checks if the denominator is zero to prevent mathematical errors.
10. then echo "Error: Division by zero is not allowed." specifies the message block to print if a user tries to divide by zero.
11. else result=$((num1 / num2)) performs the safe integer division and prints the final quotient if the denominator is valid.
12. fi closes out the internal zero check validation block.
13. *) echo "Invalid choice!" acts as a fallback handler that catches any numbers or characters outside the 1 to 4 option range.
14. esac closes out the multi-option case conditional selection structure.

Execution Steps:

1. Open the file using vi calculator.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 calculator.bash.
5. Run the script using ./calculator.bash

```
[root@ip-172-31-43-150 bash_scripts]# vi calculator.bash
[root@ip-172-31-43-150 bash_scripts]# chmod 755 calculator.bash
[root@ip-172-31-43-150 bash_scripts]# ./calculator.bash
Enter the first number:
10
Enter the second number:
20
Choose an operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
1
Result = 30
[root@ip-172-31-43-150 bash_scripts]# ./calculator.bash
Enter the first number:
2
Enter the second number:
3
Choose an operation:
1. Addition
2. Subtraction
3. Multiplication
4. Division
3
Result = 6
[root@ip-172-31-43-150 bash_scripts]#
```

8. Create a bash script to check if a directory exists, and if not, create it.

```bash
#!/bin/bash

echo "Enter the directory name:"
read dir_name

echo

if find / -type d -name "$dir_name" -print -quit 2>/dev/null | grep -q .
then
    echo "Directory exists"
else
    echo "Directory does not exist"
    echo "Enter the name of the directory you want to create:"
    read dir_name

    mkdir "$dir_name"

    echo
    echo "New directory $dir_name has been created"
fi
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.

2. echo "Enter the directory name:" displays a message on the screen prompting the user to enter a directory name.
3. read dir_name captures the user input from the keyboard and stores it into a variable named dir_name.
4. echo prints a blank line for visual spacing in the terminal window.
5. if find / -type d -name "$dir_name" -print -quit 2>/dev/null | grep -q . starts a conditional control block that searches the entire system for a directory matching the input name, hides errors, and checks if a result exists.
6. then echo "Directory exists" specifies the message block to print if the directory is found on the system.
7. else echo "Directory does not exist" handles the alternative scenario when no matching directory is found.
8. echo "Enter the name of the directory you want to create:" prompts the user to provide a name for the new folder.
9. read dir_name captures the new directory name input and stores it into the dir_name variable.
10. mkdir "$dir_name" executes the system command to create the new directory with the specified name.
11. echo prints an empty line for terminal formatting.
12. echo "New directory $dir_name has been created" outputs a final confirmation message confirming the folder creation.
13. fi closes out the conditional block structure.

Execution Steps:

1. Open the file using vi check_dir.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 check_dir.bash.
5. Run the script using ./check_dir.bash



9. Create a bash script to delete the last 3 lines of a file.

```
#!/bin/bash

echo "Enter the file name:"
read file_name

if [ -f "$file_name" ]
then
    sed -i '$d' "$file_name"
    sed -i '$d' "$file_name"
    sed -i '$d' "$file_name"

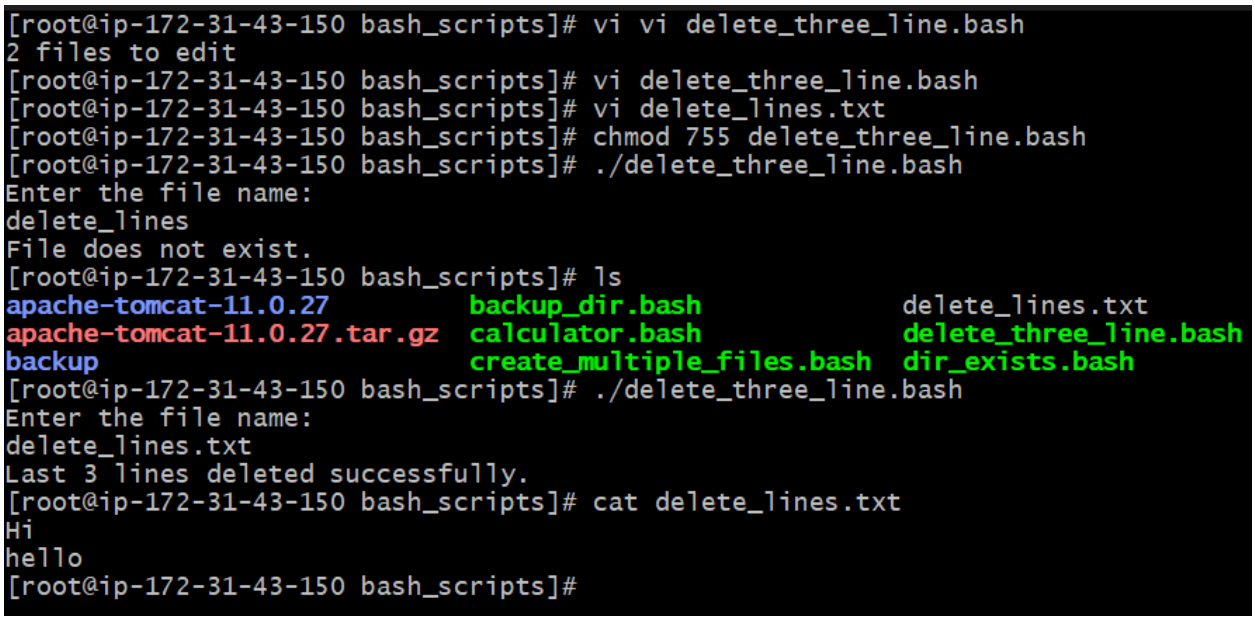
    echo "Last 3 lines deleted successfully."
else
    echo "File does not exist."
fi
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. echo "Enter the file name:" prints a message prompting the user to provide the name of the file they want to modify.
3. read file_name captures the terminal keyboard input and stores it into a variable named file_name.
4. if [-f "$file_name"] starts a conditional block to verify whether the specified file exists and is a regular file.
5. then opens the command execution path if the target file is found on the system.
6. sed -i '$d'"file_name" runs a stream editor command to delete the current last line of the file directly in place.
7. sed -i '$d'"file_name" repeats the in-place deletion command to remove the new trailing line of the file.
8. sed -i '$d'"file_name" executes the deletion command a third time to complete the removal of the final three lines.
9. echo "Last 3 lines deleted successfully." outputs a terminal message confirming that the text lines were removed.
10. else defines the fallback operation block if the file condition check fails.
11. echo "File does not exist." prints an error alert stating that the targeted file name could not be resolved.
12. fi closes out the file check conditional block structure.

Execution Steps:

1. Open the file using vi delete_three_line.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 delete_three_line.bash.
5. Run the script using ./delete_three_line.bash.
6. When prompted, enter delete_lines.txt as the file name to execute the changes.
7. Verify the remaining contents of the file using cat delete_lines.txt



10. Create a bash script to monitor CPU usage and send an email alert if it exceeds 80%

```
#!/bin/bash

CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

THRESHOLD=80
EMAIL="your-email@example.com"

if (( ${CPU%.*} > THRESHOLD ))
then
    echo "CPU usage is ${CPU}% on $(hostname)" | mail -s "High CPU Usage Alert" "$EMAIL"
    echo "CPU usage is ${CPU}%. Email notification sent."
else
    echo "CPU usage is ${CPU}%. CPU usage is normal."
fi
~
~
```

.

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}') checks the current system performance by running top in batch mode once, extracting the CPU usage line, and subtracting the idle percentage from 100 to get the active usage.
3. THRESHOLD=80 sets a variable named THRESHOLD to define the maximum safe CPU percentage limit before triggering an alert.
4. EMAIL="your-email@example.com" stores the destination email address where notifications will be sent in a variable named EMAIL.
5. if ((${CPU%.*} > THRESHOLD)) starts a conditional control block that strips decimal values from the calculated CPU usage and checks if the remaining integer is greater than the target threshold limit.
6. then opens the command execution path if the measured CPU load crosses the defined threshold limit.
7. echo "CPU usage is ${CPU}% on $(hostname)"|mail -s "High CPU Usage Alert"" EMAIL" pipes an automated diagnostic message containing the server hostname into the mail utility to deliver a high usage email alert.
8. echo "CPU usage is ${CPU}%. Email notification sent." prints a log message directly on the local terminal screen confirming that the alert email was dispatched.
9. else defines the alternative script execution logic if the current system load is within safe boundaries.
10. echo "CPU usage is ${CPU}%. CPU usage is normal." outputs a reassuring status report stating that server utilization is currently within normal limits.
11. fi closes out the performance monitoring conditional validation block structure.

Execution Steps:

1. Open the file using vi cpu-monitor.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 cpu-monitor.bash.
5. Run the script using ./cpu-monitor.bash.

```
[root@ip-172-31-45-159 saam]# cd scripts
[root@ip-172-31-45-159 scripts]# vi cpu-monitor.bash
[root@ip-172-31-45-159 scripts]# chmod 755 cpu-monitor.bash
[root@ip-172-31-45-159 scripts]# ./cpu-monitor.bash
CPU usage is 3.2%. CPU usage is normal.
[root@ip-172-31-45-159 scripts]#
```

11. Bash script to monitor disk space and if it is more than 80% then send email notification.

```
#!/bin/bash

DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

THRESHOLD=80
EMAIL="ayesha.erml@gmail.com"

if [ "$DISK" -gt "$THRESHOLD" ]
then
    echo "Disk usage is ${DISK}% on $(hostname)" | mail -s "High Disk Usage Alert" "$EMAIL"
    echo "Disk usage is ${DISK}%. Email notification sent."
else
    echo "Disk usage is ${DISK}%. Disk usage is normal."
fi
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%') checks the root directory storage by fetching system disk consumption, extracting the fifth column text output from the second data line, and stripping away the percentage character.
3. THRESHOLD=80 defines a variable named THRESHOLD to establish the percentage ceiling limit allowed before triggering a system alert.
4. EMAIL="ayesha.erm1@gmail.com" stores the specific administrator destination email mailbox address into a variable named EMAIL.

5. if ["$DISK" -gt "THRESHOLD"] initiates a conditional evaluation statement block to determine if the calculated storage value number is greater than the defined alert limit.
6. then starts the execution workflow sequence if the storage metric surpasses your maximum threshold boundary line.
7. echo "Disk usage is ${DISK}% on $(hostname)" | mail -s "HighDiskUsageAlert" "EMAIL" strings together the custom alert context alongside server host machine details and hands it off to the local mail server application to transmit an outgoing notification.
8. echo "Disk usage is ${DISK}%. Email notification sent." outputs a diagnostic message on the server interface indicating an alert status event was dispatched.
9. else defines the fallback operations logic path when storage levels remain safely under the target alarm limit.
10. echo "Disk usage is ${DISK}%. Disk usage is normal." writes a standard confirmation line to the system terminal displaying current occupancy parameters.
11. fi finishes up the target data storage monitoring evaluation block configuration structure.

Execution Steps:

1. Open the file using vi disk-space.bash.
2. Enter the script shown in your screenshot.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 disk-space.bash.
5. Run the script using ./disk-space.bash.

```
[root@ip-172-31-45-159 scripts]# vi disk-space.bash
[root@ip-172-31-45-159 scripts]# chmod 755 disk-space.bash
[root@ip-172-31-45-159 scripts]# ./disk-space.bash
Disk usage is 25%. Disk usage is normal.
[root@ip-172-31-45-159 scripts]#
```

12. Bash script to monitor memory and if it is more than 80% then send email notification

```
#!/bin/bash

MEMORY=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')

THRESHOLD=80
EMAIL="your-email@example.com"

if [ "$MEMORY" -gt "$THRESHOLD" ]
then
    echo "Memory usage is ${MEMORY}% on $(hostname)" | mail -s "High Memory Usage Alert" "$EMAIL"
    echo "Memory usage is ${MEMORY}%. Email notification sent."
else
    echo "Memory usage is ${MEMORY}%. Memory usage is normal."
fi
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.
2. FREE_MEM=$(free | grep Mem | awk '{print $4}') extracts the currently available free physical memory space from the system information using the free command.
3. TOTAL_MEM=$(free | grep Mem | awk '{print $2}') extracts the total amount of physical memory installed on the machine.
4. USED_MEM=$((TOTAL_MEM - FREE_MEM)) calculates the amount of memory currently occupied by subtracting free memory from total memory.
5. MEM_PERCENT=$((USED_MEM * 100 / TOTAL_MEM)) calculates the final memory usage percentage by scaling the used memory values.
6. THRESHOLD=80 establishes a variable named THRESHOLD to set the maximum allowed memory usage limit before an alert triggers.
7. EMAIL="your-email@example.com" stores the target administrator mailbox address inside a variable named EMAIL.
8. if ["$MEM_PERCENT" -gt "THRESHOLD"] initiates a conditional evaluation statement block to check if the calculated memory percentage is greater than the alert limit.
9. then opens the command path if the current memory load surpasses the maximum threshold boundary line.
10. echo "Memory usage is high: ${MEM_PERCENT}% on $(hostname)" | mail -s "High Memory Usage Alert" "EMAIL" sends an automated diagnostic alert text containing the server name through the mail command.
11. echo "Memory usage is ${MEM_PERCENT}%. Email notification sent." prints a log message directly to the screen confirming the notification process completed.
12. else defines the fallback operation block if the memory consumption is safely below the limit.
13. echo "Memory usage is ${MEM_PERCENT}%. Memory usage is normal." writes a standard statement line to the terminal confirming steady operational levels.

14. fi closes out the systems performance validation block configuration structure.

Execution Steps:

1. Open the file using vi memory-monitor.bash.
2. Enter the script logic into the text editor utility.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 memory-monitor.bash.
5. Run the script using ./memory-monitor.bash

```
[root@ip-172-31-45-159 scripts]# vi monitor-memory.bash
[root@ip-172-31-45-159 scripts]# chmod 755 monitor-memory.bash
[root@ip-172-31-45-159 scripts]# ./monitor-memory.bash
Memory usage is 26%. Memory usage is normal.
[root@ip-172-31-45-159 scripts]#
```

12. Bash script to monitor memory and if it is more than 80% then send email notification.

```
#!/bin/bash

MEMORY=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')

THRESHOLD=80
EMAIL="your-email@example.com"

if [ "$MEMORY" -gt "$THRESHOLD" ]
then
    echo "Memory usage is ${MEMORY}% on $(hostname)" | mail -s "High Memory Usage Alert" "$EMAIL"
    echo "Memory usage is ${MEMORY}%. Email notification sent."
else
    echo "Memory usage is ${MEMORY}%. Memory usage is normal."
fi
~
```

Explanation of the Code:

1. #!/bin/bash specifies that the script should be run using the Bash shell.

2. FREE_MEM=$(free | grep Mem | awk '{print $4}') extracts the currently available free physical memory space from the system information using the free command.
3. TOTAL_MEM=$(free | grep Mem | awk '{print $2}') extracts the total amount of physical memory installed on the machine.
4. USED_MEM=$((TOTAL_MEM - FREE_MEM)) calculates the amount of memory currently occupied by subtracting free memory from total memory.
5. MEM_PERCENT=$((USED_MEM * 100 / TOTAL_MEM)) calculates the final memory usage percentage by scaling the used memory values.
6. THRESHOLD=80 establishes a variable named THRESHOLD to set the maximum allowed memory usage limit before an alert triggers.
7. EMAIL="your-email@example.com" stores the target administrator mailbox address inside a variable named EMAIL.
8. if ["$MEM_{P}ERCENT" - gt"THRESHOLD"] initiates a conditional evaluation statement block to check if the calculated memory percentage is greater than the alert limit.
9. then opens the command path if the current memory load surpasses the maximum threshold boundary line.
10. echo "Memory usage is high: ${MEM_PERCENT}% on $(hostname)"|mail - s"HighMemoryUsageAlert""EMAIL" sends an automated diagnostic alert text containing the server name through the mail command.
11. echo "Memory usage is ${MEM_PERCENT}%. Email notification sent." prints a log message directly to the screen confirming the notification process completed.
12. else defines the fallback operation block if the memory consumption is safely below the limit.
13. echo "Memory usage is ${MEM_PERCENT}%. Memory usage is normal." writes a standard statement line to the terminal confirming steady operational levels.
14. fi closes out the systems performance validation block configuration structure.

Execution Steps:

1. Open the file using vi memory-monitor.bash.
2. Enter the script logic into the text editor utility.
3. Press Esc, type :wq, and press Enter to save and exit.
4. Give execution permission to the script using chmod 755 memory-monitor.bash.
5. Run the script using ./memory-monitor.bash.

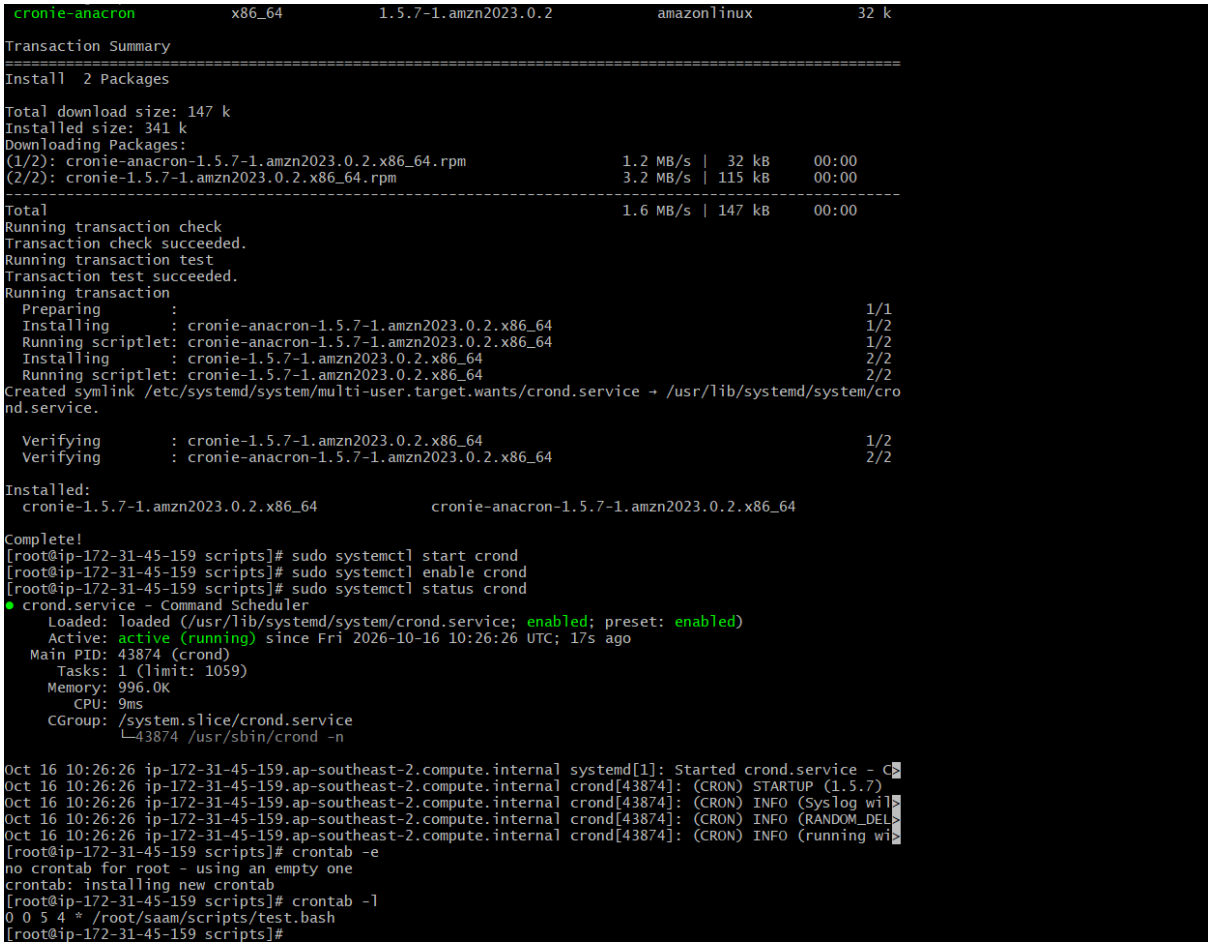
```
[root@ip-172-31-45-159 scripts]# vi monitor-memory.bash
[root@ip-172-31-45-159 scripts]# chmod 755 monitor-memory.bash
[root@ip-172-31-45-159 scripts]# ./monitor-memory.bash
Memory usage is 26%. Memory usage is normal.
[root@ip-172-31-45-159 scripts]#
```

Exercise: Crontab Entries

1. April 5th Midnight

```
#!/bin/bash

echo "Cron job executed on $(date)" >> /home/ayesha/cron_output.txt
~
```



Steps — April 5th at Midnight

1. I checked the operating system of my EC2 instance.
2. I checked whether the crontab command was available.
3. I installed the required cron package if crontab was not available.
4. I started the cron service.
5. I enabled the cron service so that it starts automatically after a reboot.
6. I created a Bash script in my scripts directory.
7. I gave execute permission to my Bash script.
8. I opened the crontab for editing.
9. I added the schedule for April 5th at midnight.
10. I specified the absolute path of my Bash script in the cron entry.
11. I saved and exited the crontab.
12. I verified the cron entry using crontab -l.
13. I configured the cron job to execute my script every year on April 5th at 12:00 AM.

2. 5th of Every November, January, June if it is a Thursday

```
0 0 5 1,6,11 4 * /root/saam/scripts/test.bash
~
~
~
~
~
~
~
```

```
[root@ip-172-31-45-159 scripts]# vi test1.bash
[root@ip-172-31-45-159 scripts]# vi test1.bash
[root@ip-172-31-45-159 scripts]# chmod 755 test1.bash
[root@ip-172-31-45-159 scripts]# corntab -e
-bash: corntab: command not found
[root@ip-172-31-45-159 scripts]# yum install cronie
Last metadata expiration check: 0:54:07 ago on Fri Oct 16 09:41:49 2026.
Package cronie-1.5.7-1.amzn2023.0.2.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-172-31-45-159 scripts]# sudo systemctl start crond
[root@ip-172-31-45-159 scripts]# sudo systemctl status crond
● crond.service - Command Scheduler
     Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-16 10:26:26 UTC; 10min ago
   Main PID: 43874 (crond)
      Tasks: 1 (limit: 1059)
     Memory: 1.0M
        CPU: 12ms
     CGroup: /system.slice/crond.service
             └─43874 /usr/sbin/crond -n

Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal systemd[1]: Started crond.service - C>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) STARTUP (1.5.7)
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (Syslog wil>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (RANDOM_DEL>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (running wi>
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# 0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
-bash: 0: command not found
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
[root@ip-172-31-45-159 scripts]#
```

1. I created my Bash script in the scripts directory.
2. I gave execute permission to the script.
3. I opened my crontab using crontab -e.
4. I added the cron schedule for the 5th of January, June, and November on Thursday.
5. I saved and exited the crontab.
6. I verified my cron entry using crontab -l.
7. I temporarily changed the schedule to every minute to test the script.

8. I checked the output to confirm that the cron job executed successfully.
9. I restored the original schedule after testing.

3. At 05 and 27th minutes of 9, 10, 11 hours every day

```
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
~
~
~
~
~
```

```
[root@ip-172-31-45-159 scripts]# corntab -e
-bash: corntab: command not found
[root@ip-172-31-45-159 scripts]# sudo systemctl start crond
sudo systemctl enable crond
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps — 05th and 27th Minutes of 9, 10, 11 Hours Every Day

1. I created my Bash script test.sh in the scripts directory.
2. I added the command to record the cron execution date and time.
3. I gave execute permission to my test.sh script.
4. I started the crond service once on my EC2 instance.
5. I enabled the crond service to start automatically after a reboot.
6. I opened my crontab using crontab -e.
7. I added the cron schedule for the 5th and 27th minutes of the 9th, 10th, and 11th hours.
8. I saved and exited the crontab.
9. I verified my cron entry using crontab -l.
10. I used the same test.sh script instead of creating a new script.
11. I configured the cron job to run every day at 09:05, 09:27, 10:05, 10:27, 11:05, and 11:27.

4. 34 minutes of 9th hour on 15th August

```
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
~
```

```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps — 34 Minutes of 9th Hour on 15th August

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the schedule for the 34th minute of the 9th hour on 15th August.
4. I saved and exited the crontab.
5. I verified the cron entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every year on 15th August at 9:34 AM.

5. Every midnight

```
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
~
```

```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps — Every Midnight

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the schedule to run the script at midnight every day.
4. I saved and exited the crontab.
5. I verified the cron entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every day at 12:00 AM (midnight).

6. Every Weekend (Saturday night 11:59 PM)

```
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
~
```

```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the schedule to run the script every Saturday at 11:59 PM.
4. I saved and exited the crontab.
5. I verified the entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every Saturday night at 11:59 PM.

7. After every reboot

```
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
@reboot /home/ayesha/scripts/test.sh
~
```

```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
@reboot /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the @reboot entry to execute my script whenever the system reboots.
4. I saved and exited the crontab.
5. I verified the entry using crontab -l.
6. I configured the cron job to run my script automatically after every system reboot.